Heaps Binários

Estruturas de Dados



UnB | CIC

Introdução

- FIFO: first-in first-out
- Uma variação importante de uma fila é chamada de fila de prioridade.
- Uma fila de prioridade age como uma fila mas a ordem lógica dos itens é determinada por sua prioridade. Os itens de maior prioridade estão na frente da fila e os itens de menor prioridade estão atrás.
- Existem algumas maneiras fáceis de implementar uma fila de prioridade usando listas. No entanto, a maneira mais adequada de implementar uma fila de prioridade é usando uma estrutura de dados chamada heap binário.

Definição

Um Heap Binário é uma estrutura de dados baseada em árvore binária completa que segue uma das seguintes propriedades:

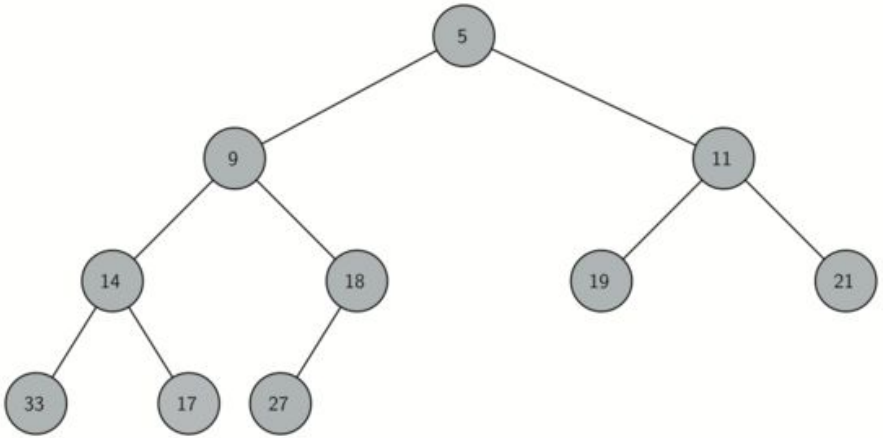
- **Max-Heap:** O valor de cada nó é maior ou igual ao de seus filhos.
- **Min-Heap:** O valor de cada nó é menor ou igual ao de seus filhos.

Propriedades

1. **Forma:** A árvore é completa, ou seja, todos os níveis são totalmente preenchidos, exceto possivelmente o último, que deve estar o mais à esquerda possível.
2. **Ordem:** Mantém uma relação de prioridade entre pais e filhos (Max-Heap ou Min-Heap).

Propriedades

1. **Forma:** A árvore é completa, ou seja, todos os níveis são totalmente preenchidos, exceto possivelmente o último, que deve estar o mais à esquerda possível.
2. **Ordem:** Mantém uma relação de prioridade entre pais e filhos (Max-Heap ou Min-Heap).



Tipos de Heap Binário

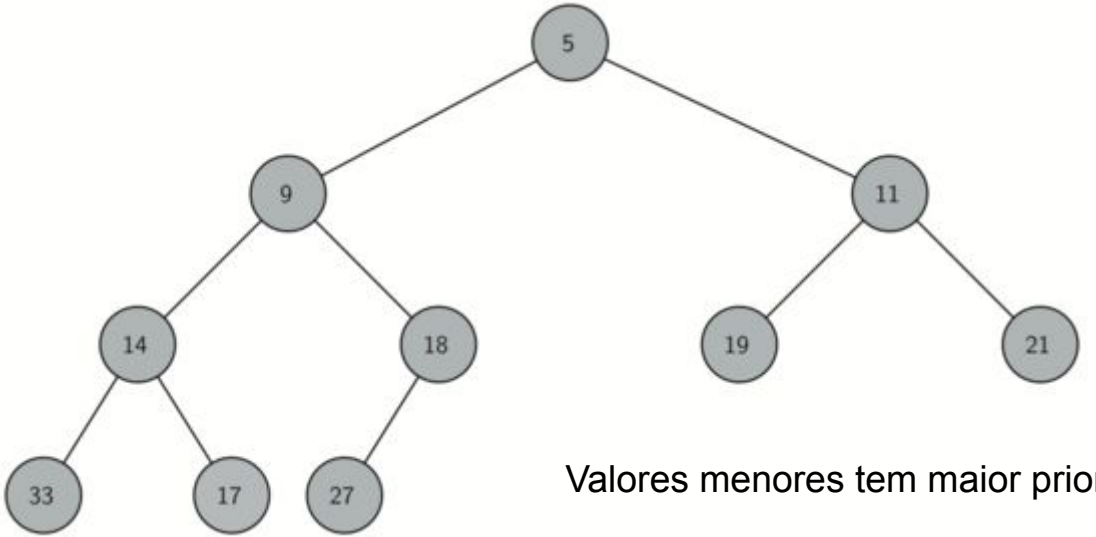
Max-Heap:

- A raiz (nó superior) contém o maior valor.
- Cada nó pai é maior ou igual aos seus filhos.

Min-Heap:

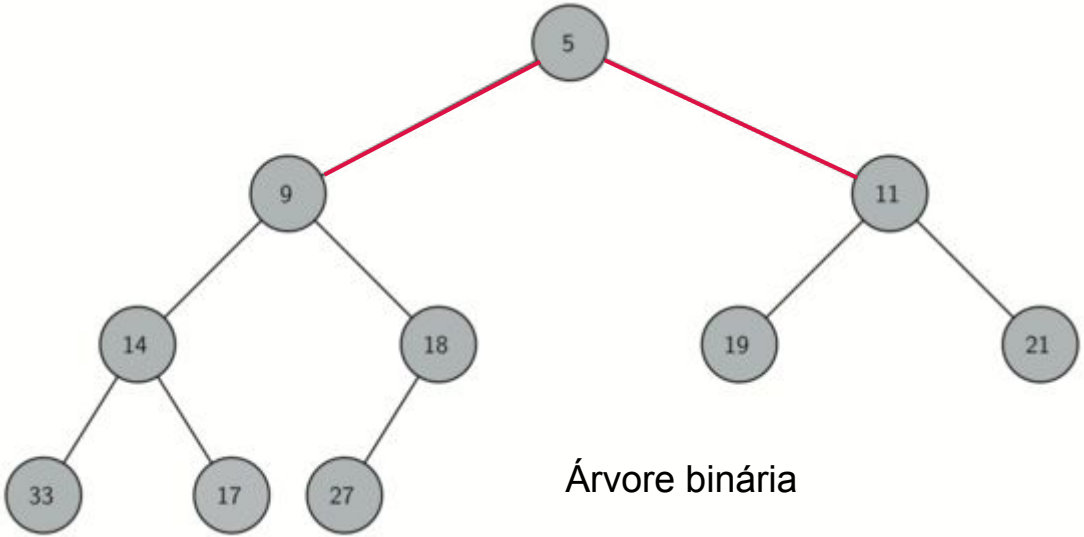
- A raiz contém o menor valor.
- Cada nó pai é menor ou igual aos seus filhos.

Heap binário (Min-Heap)



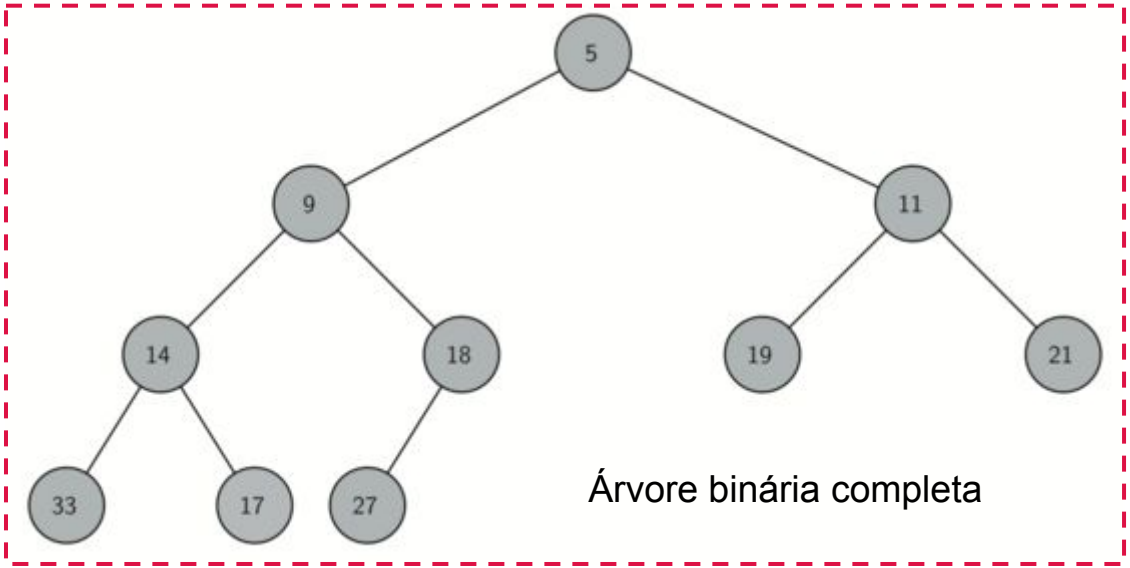
Valores menores tem maior prioridade

Heap binário (Min-Heap)



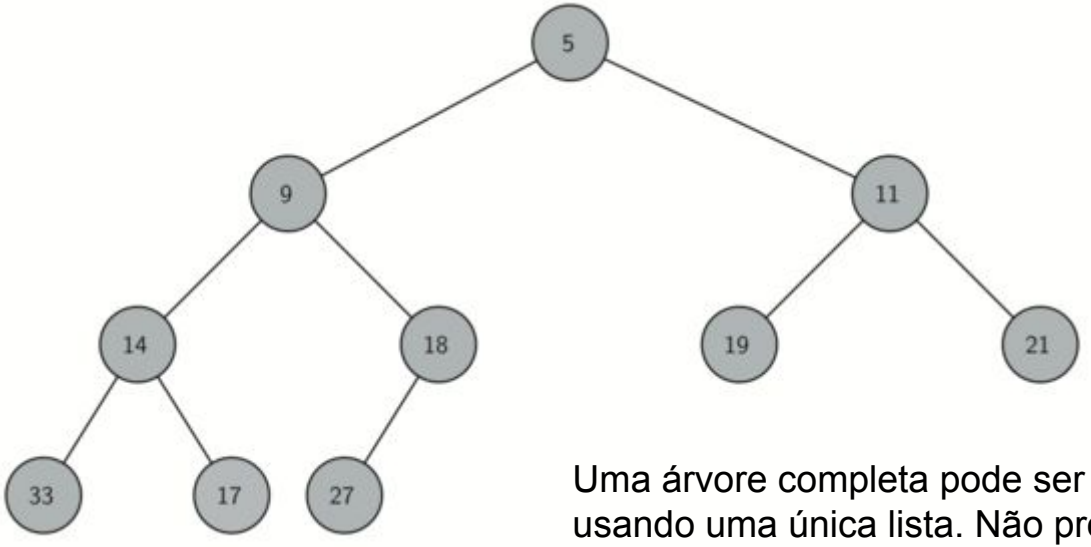
Árvore binária

Heap binário (Min-Heap)



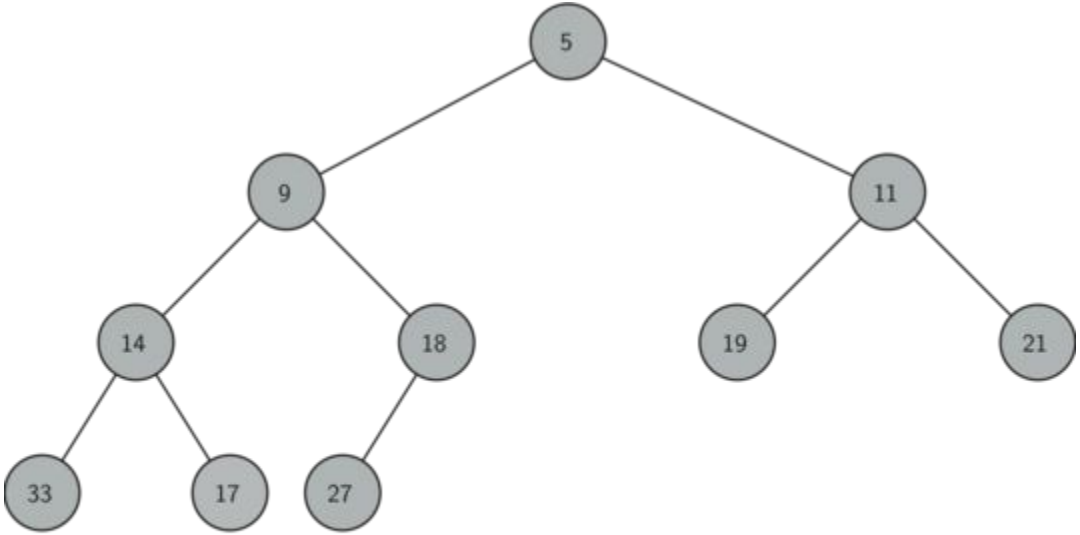
Uma árvore binária completa é uma árvore binária em que todos os níveis, exceto possivelmente o último, estão completamente preenchidos, e todos os nós estão o mais à esquerda possível.

Heap binário (Min-Heap)



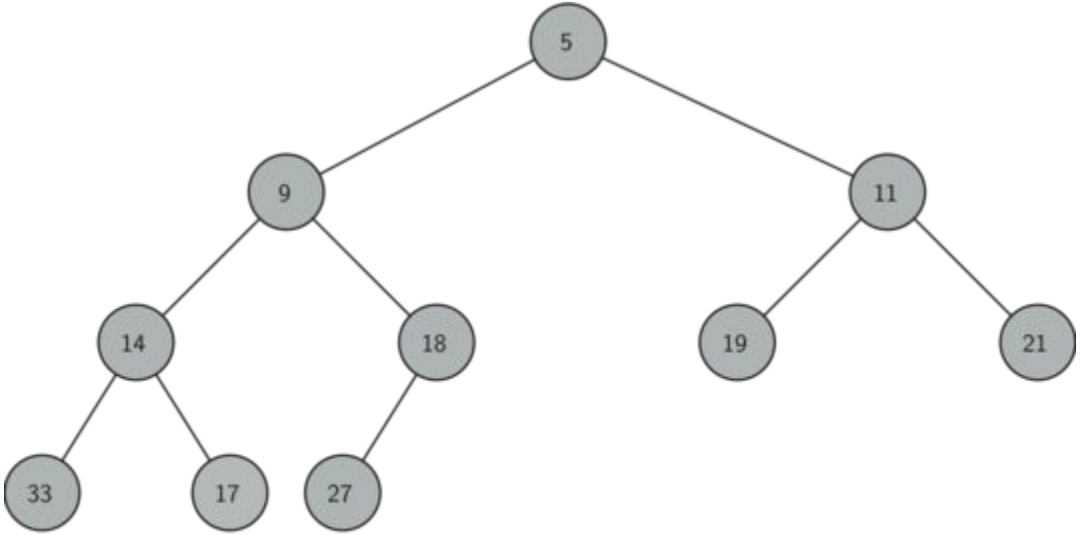
Uma árvore completa pode ser representada usando uma única lista. Não precisamos usar nós e referências ou mesmo listas de listas.

Heap binário (Min-Heap)



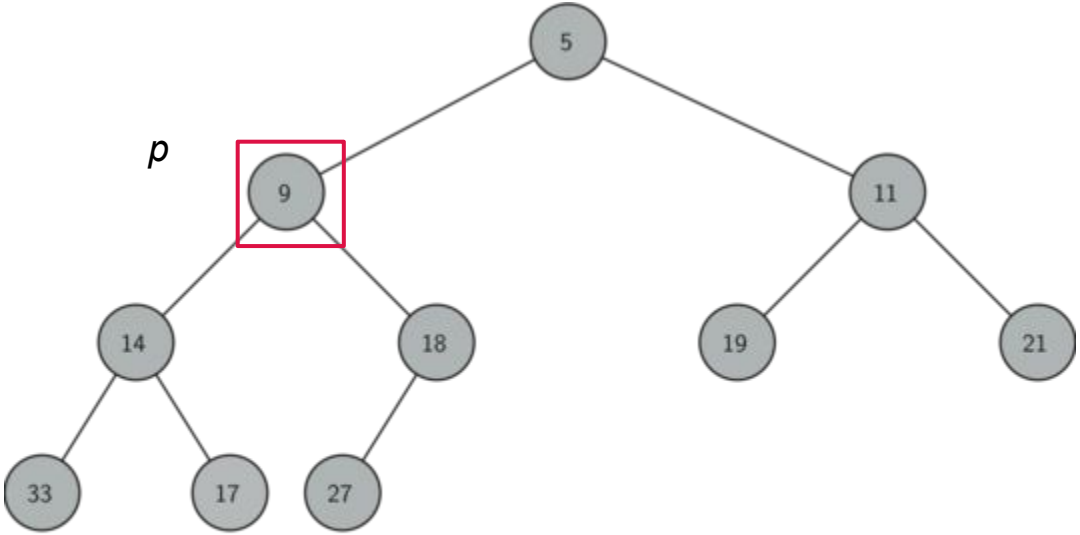
0	5	9	11	14	18	19	21	33	17	27	
0	1	2	3	4	5	6	7	8	9	10	11

Heap binário (Min-Heap)

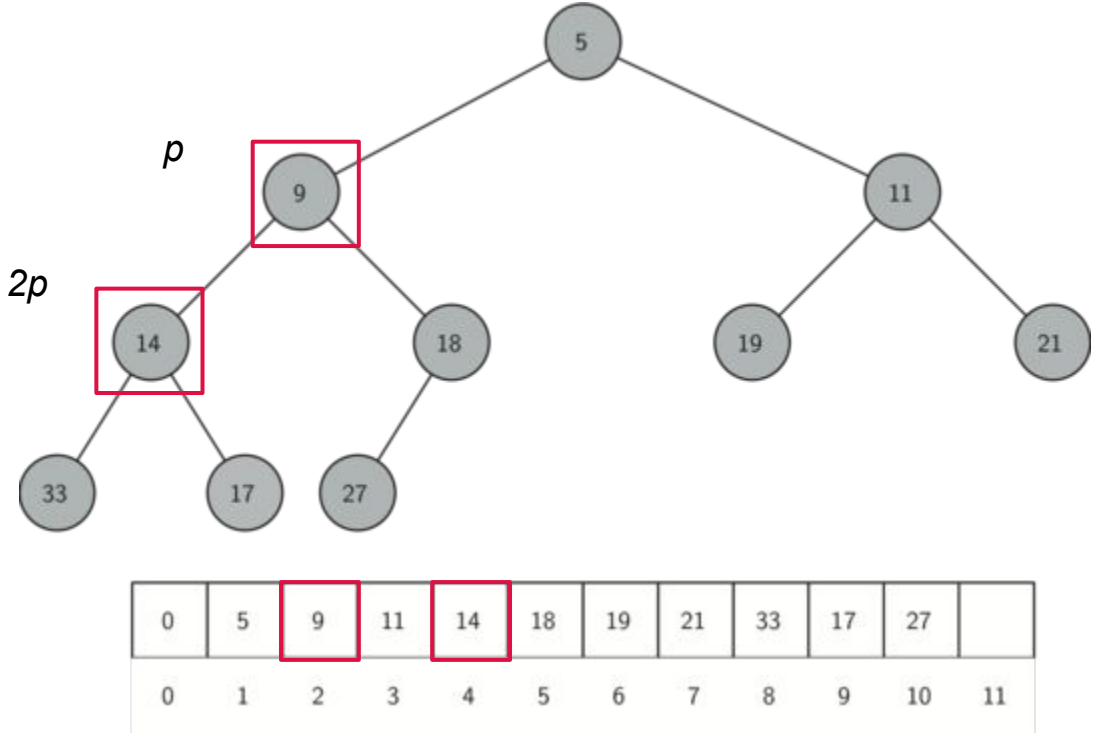


0	5	9	11	14	18	19	21	33	17	27	
0	1	2	3	4	5	6	7	8	9	10	11

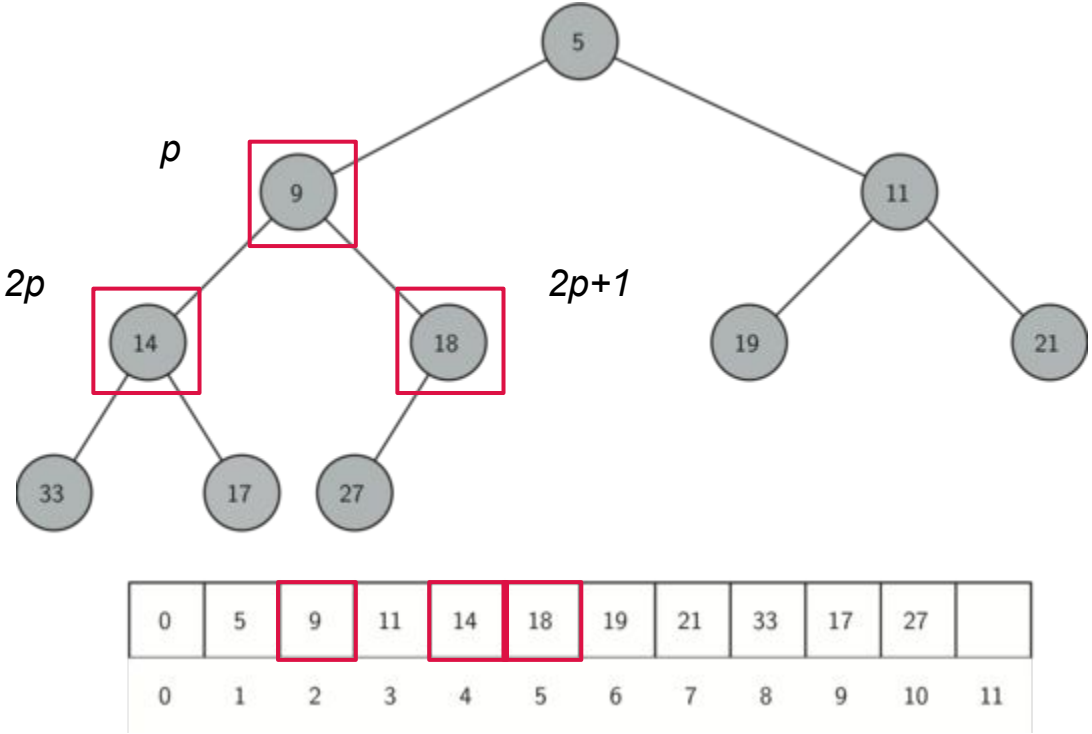
Heap binário (Min-Heap)



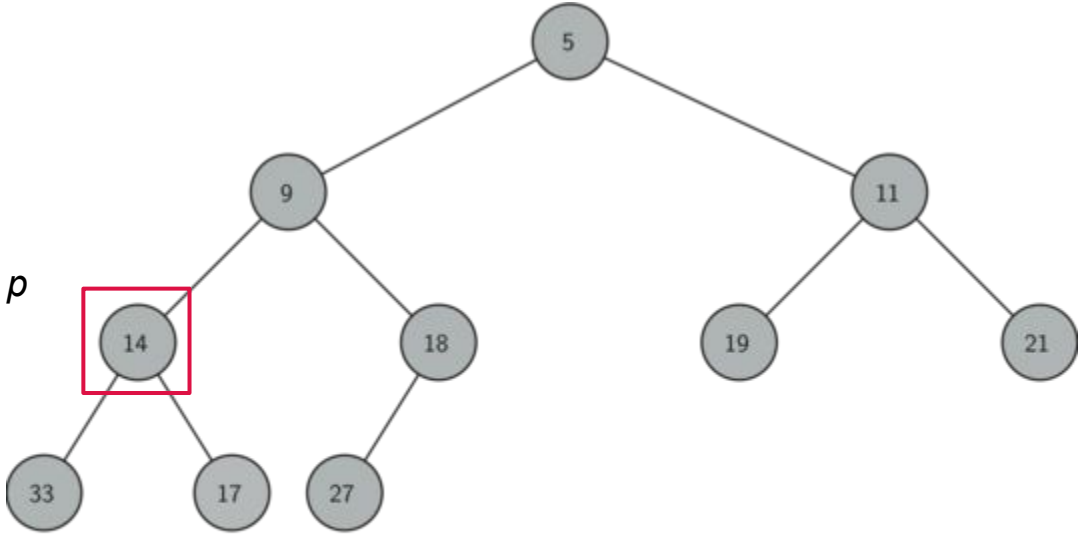
Heap binário (Min-Heap)



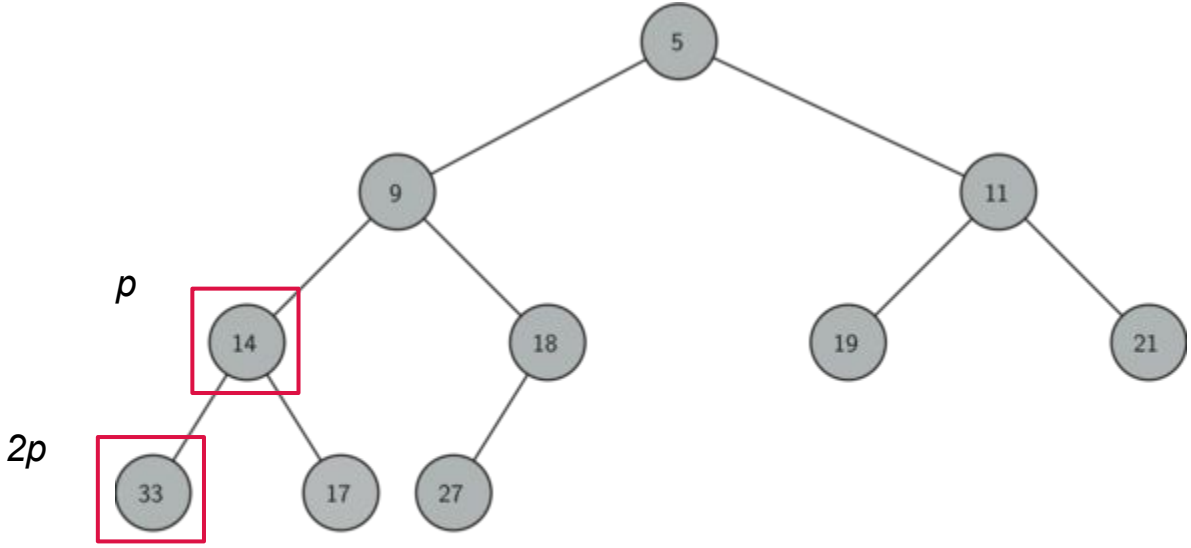
Heap binário (Min-Heap)



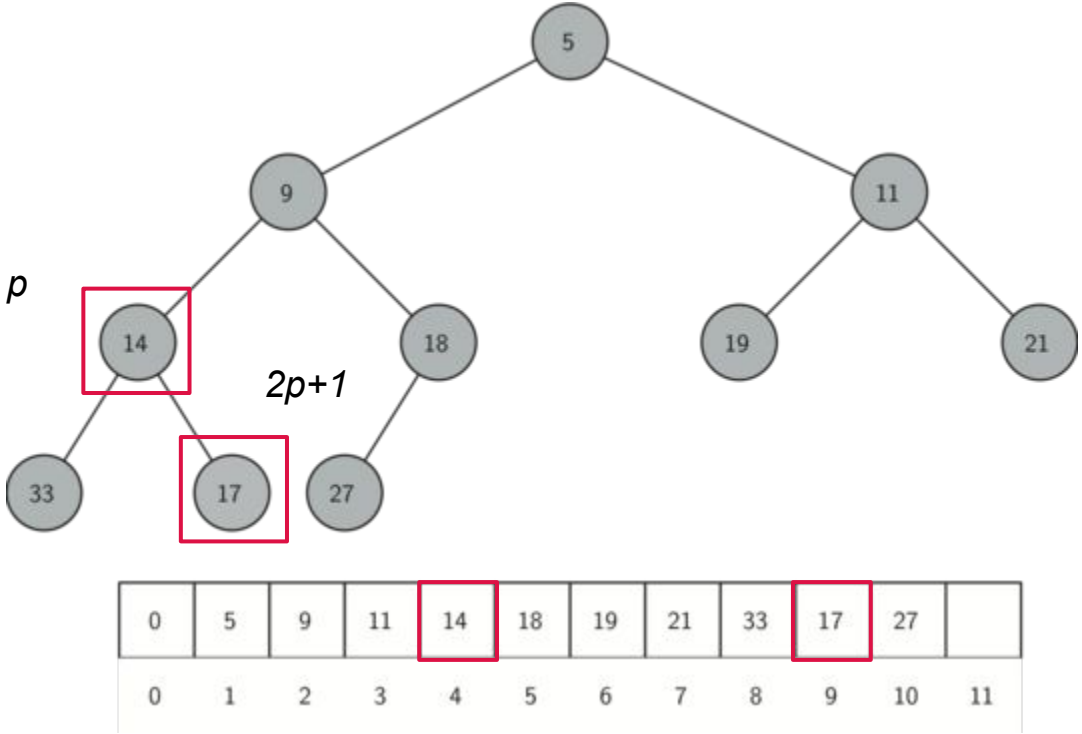
Heap binário (Min-Heap)



Heap binário (Min-Heap)



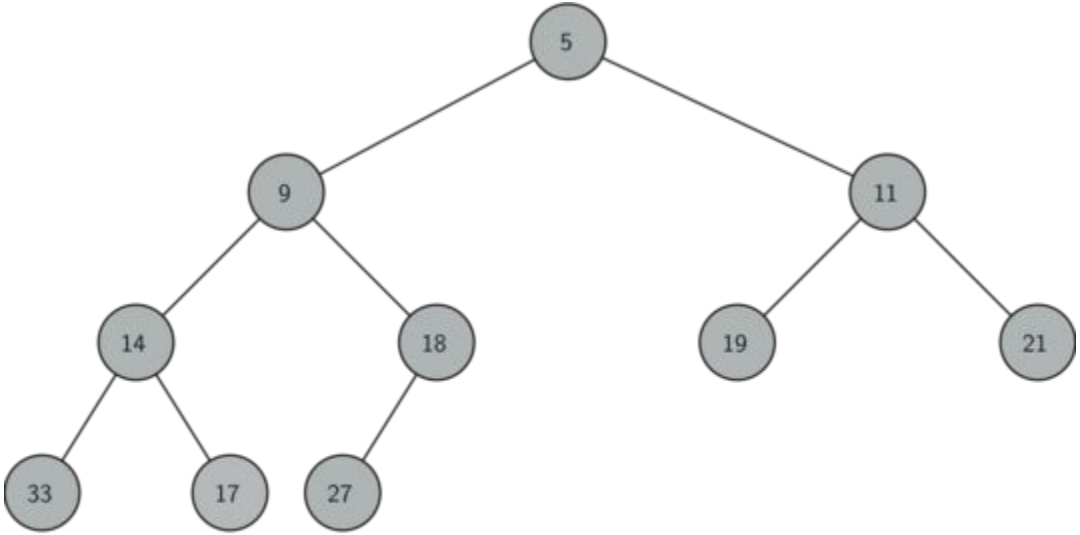
Heap binário (Min-Heap)



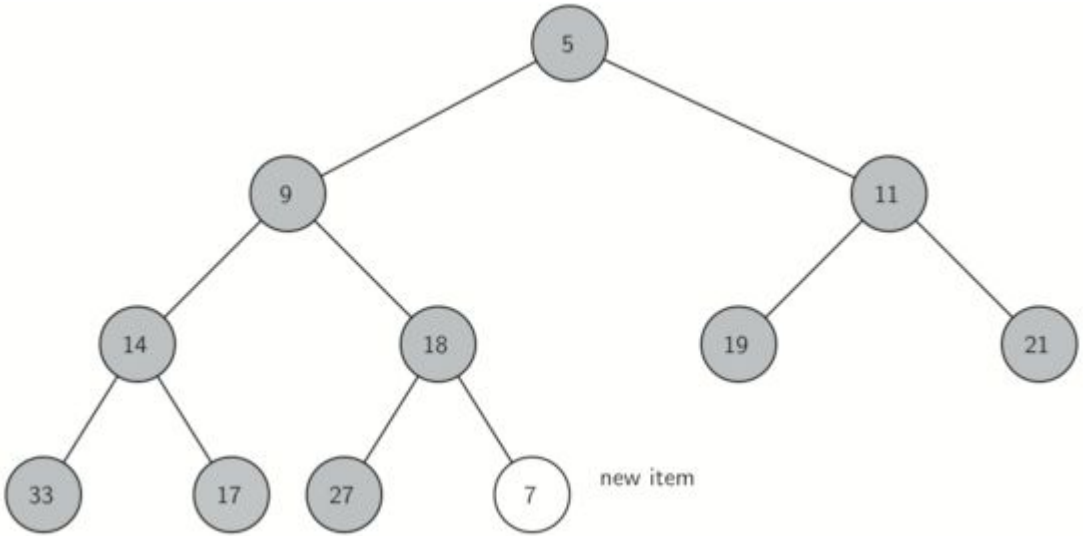
Heap binário

```
class BinHeap:
    def __init__(self):
        self.heapList = [0]
        self.currentSize = 0
```

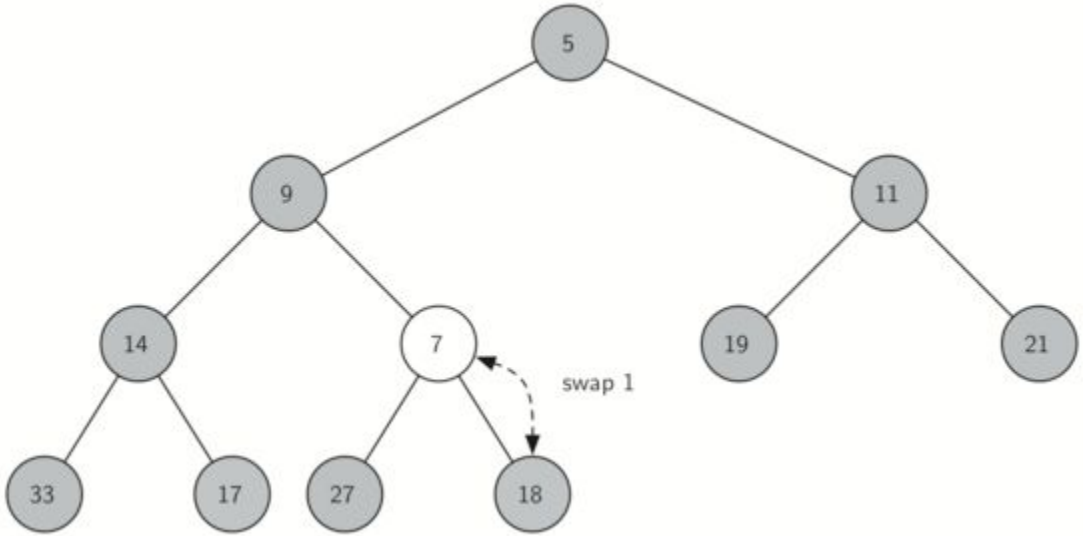
Inserção no heap binário



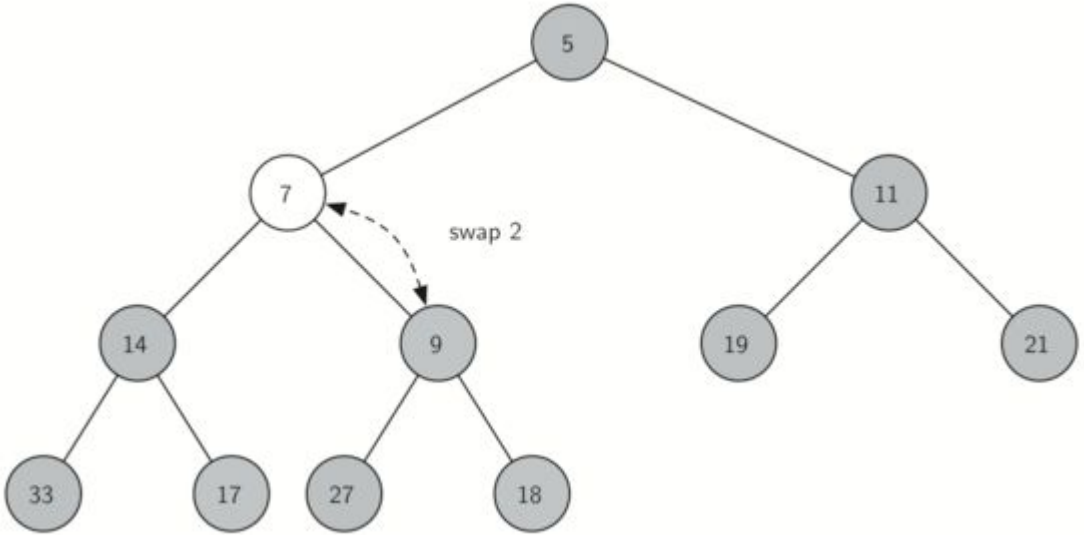
Inserção no heap binário



Inserção no heap binário



Inserção no heap binário

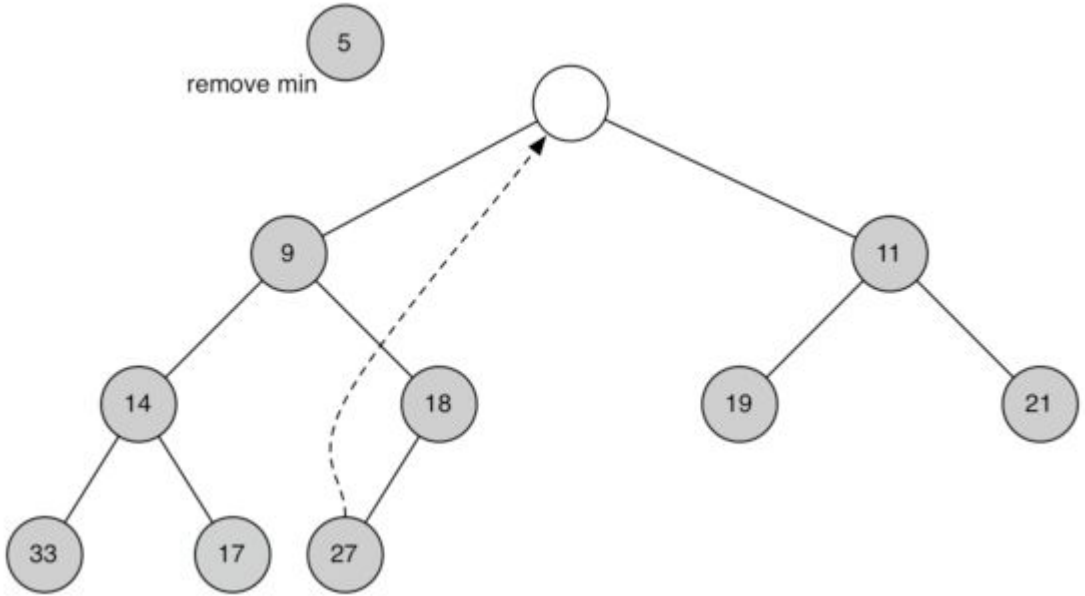


Inserção no heap binário

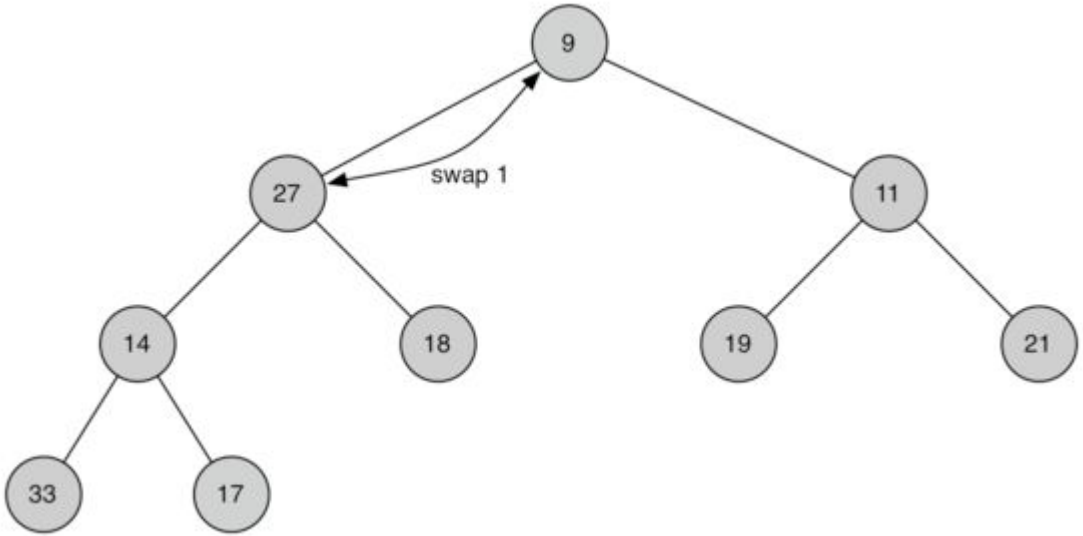
```
def insert(self, k):
    self.heapList.append(k)
    self.currentSize = self.currentSize + 1
    self.percUp(self.currentSize)


def percUp(self, i):
    while i // 2 > 0:
        if self.heapList[i] < self.heapList[i // 2]:
            tmp = self.heapList[i // 2]
            self.heapList[i // 2] = self.heapList[i]
            self.heapList[i] = tmp
        i = i // 2
```

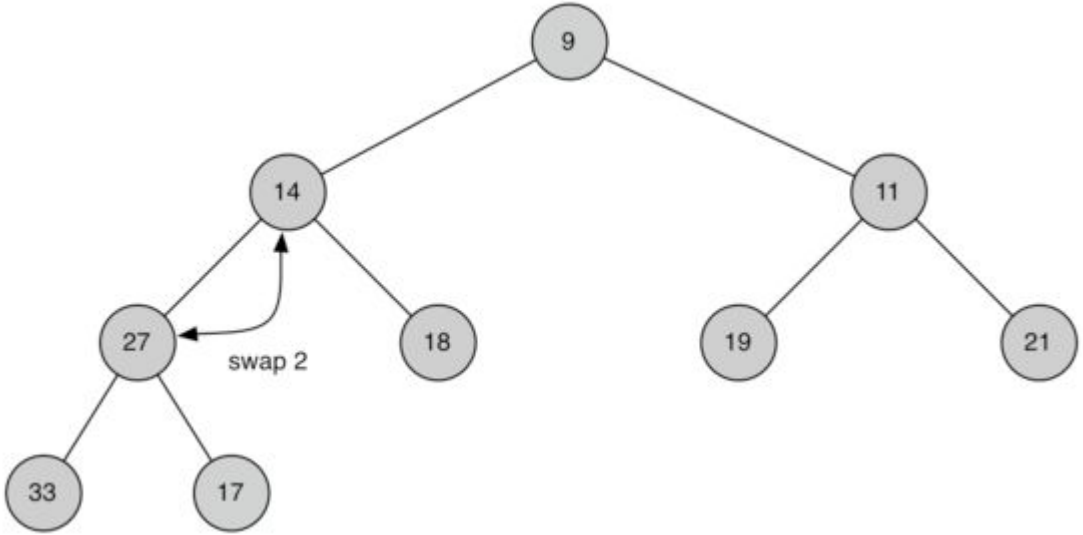
Deleta o valor mínimo no heap binário



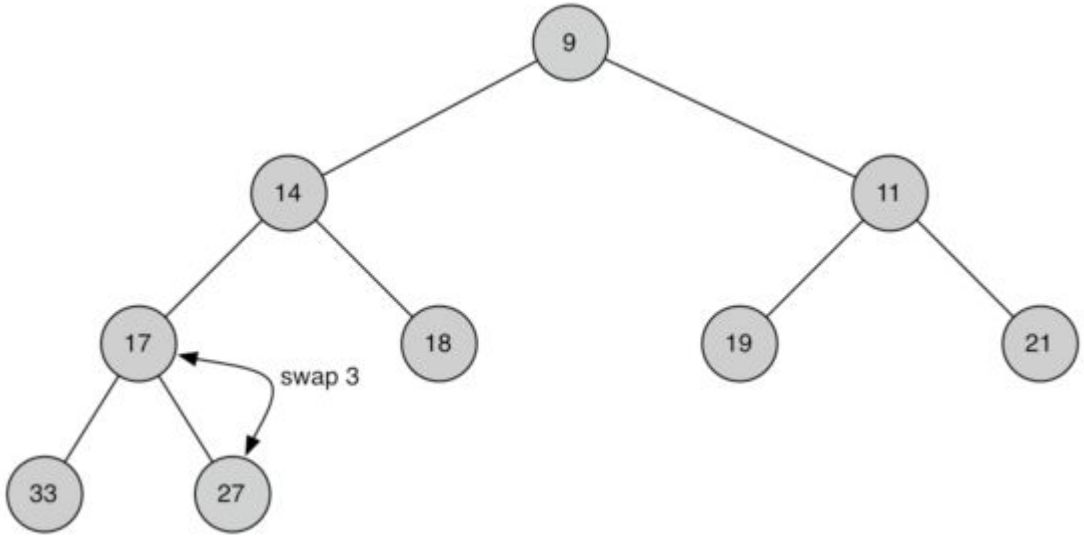
Deleta o valor mínimo no heap binário



Deleta o valor mínimo no heap binário



Deleta o valor mínimo no heap binário



Deleta o valor mínimo no heap binário

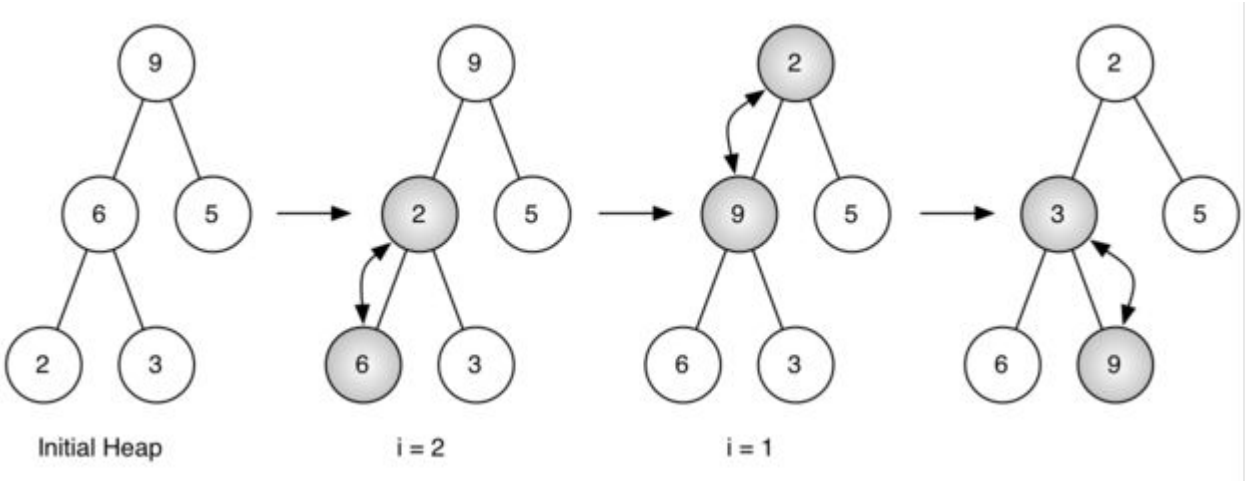
```
def delMin(self):
    retval = self.heapList[1]
    self.heapList[1] = self.heapList[self.currentSize]
    self.currentSize = self.currentSize - 1
    self.heapList.pop()
    self.percDown(1)
    return retval


def percDown(self, i):
    while (i * 2) <= self.currentSize:
        mc = self.minChild(i)
        if self.heapList[i] > self.heapList[mc]:
            tmp = self.heapList[i]
            self.heapList[i] = self.heapList[mc]
            self.heapList[mc] = tmp
         i = mc
```

Deleta o valor mínimo no heap binário

```
def minChild(self,i):
    if i * 2 + 1 > self.currentSize:
        return i * 2
    else:
        if self.heapList[i*2] < self.heapList[i*2+1]:
            return i * 2
        else:
            return i * 2 + 1
```

Constrói heap binário



Constrói heap binário

```
def buildHeap(self, alist):
    i = len(alist) // 2
    self.currentSize = len(alist)
    self.heapList = [0] + alist[:]
    while (i > 0):
        self.percDown(i)
        i = i - 1
```

Árvore Binária de Busca

https://youtube.com/playlist?list=PLm6RIeTHkGXiIU7axErc38DxInaicWuUI



Resumo

- Fila de prioridade é uma fila cujos elementos têm prioridades (que determinam sua ordem).
- Fila de prioridade pode ser mapeada para uma estrutura de árvore binária.
- Podemos utilizar listas para representar árvores binárias completas.
- Operações de adição e remoção em uma fila de prioridade podem ser mais eficientes utilizando heap binário.